

De la Nube a la FPGA: Introducción Práctica a RISC-V con Quintauris

Ignacio Ruiz · Session 5
Testing & Validation Staff Engineer



·QUINTAURIS

Agenda

SEMANA 1

LUN 7
17:00–19:00

1. Fundamentos de RISC-V: de la arquitectura al ecosistema
Quintauris · Ángel Berrio

MAR 8
17:00–19:00

2. Presentación de Quintauris: visión, modelo de negocio y productos
Quintauris · Ángel Berrio

MIÉ 9
17:00–19:00

3. Explorando la plataforma Quintauris, Tools and Infra — incluyendo Testing, Benchmarking y QNTAF (I)
Quintauris · Enrique Pallares

JUE 10
17:00–19:00

4. Explorando la plataforma Quintauris, Tools and Infra — incluyendo Testing, Benchmarking y QNTAF (II)
Quintauris · Enrique Pallares

SEMANA 2

2 Semanas - 8 sesiones

LUN 14
16:30–19:30

5. Verificación y validación: del requisito a la evidencia, en software y en silicio
Quintauris · Ignacio Ruiz

MAR 15
16:30–19:30

6. Parte I: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA
UGR & Quintauris · José Luis Martínez / UGR

MIÉ 16
16:30–19:30

7. Parte II: De la nube a la FPGA — flujo de despliegue de una IP en la FPGA
UGR & Quintauris · José Luis Martínez / UGR

JUE 17
16:00–20:00

8. Hackathon final: integración de conceptos y solución de un reto práctico
Quintauris · José Luis Martínez, con mentores de UGR

The Quintauris Team



Ángel Berrio

Chief Product Officer

Sessions 1 and 2



Enrique Pallares

Head of Professional
Services

Sessions 3 and 4



Ignacio Ruiz

Testing & Validation
Staff Engineer

**Session 5 and
Hackathon**



José Luis Martínez

Application Engineer

**Sessions 6, 7 and
Hackathon**

Session 5

Verification & Validation — why we test, how we test software, how we test silicon

Agenda · 01



- 01 Why testing matters
- 02 Software testing: the essentials
- 03 How we do it: the QNTAF strategy
- 04 Hardware testing: the silicon lifecycle
- 05 Wrap-up & discussion



Three expensive lessons



1999 · SPACECRAFT

\$327.6 M

Mars Climate Orbiter — lost at Mars orbit insertion

1996 · LAUNCHER

>\$370 M

Ariane 5 flight 501 — lost 37 s after liftoff

1994 · PROCESSOR

\$475 M

Intel Pentium FDIV — every chip replaced on request

None was a failure of talent. All three were failures of verification.

Mars Climate Orbiter, 1999



Every unit passed its tests. The interface between them was never tested.

ALTITUDE AT MARS ORBIT INSERTION



LESSON · INTEGRATION TESTING

THE BUG

×4.45

pound-force · seconds delivered
newton · seconds expected

THE SPEC

The interface specification required SI units.

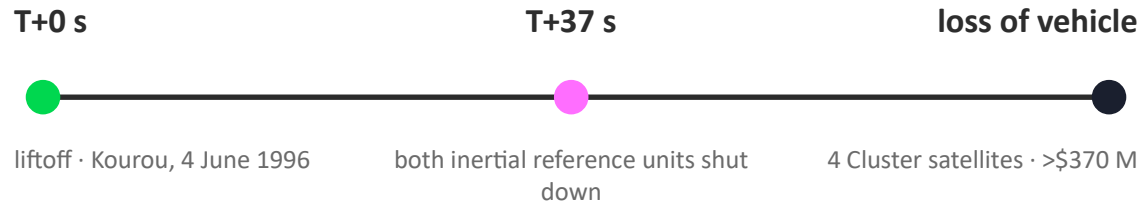
THE WARNING

Two navigators flagged the trajectory anomaly during the cruise. Dismissed — they had not filed the right form.

Ariane 5 flight 501, 1996



Verified — against the wrong reality.



64-bit float
horizontal bias



16-bit integer
OVERFLOW

Code inherited from Ariane 4 — where this value could never overflow. It was still running 37 s after liftoff, doing nothing useful.

THE INQUIRY BOARD

“The specification of the inertial reference system and the tests performed at equipment level did not specifically include the Ariane 5 trajectory data.”

REDUNDANCY DID NOT HELP

The backup unit ran the same software and failed the same way. Redundancy covers random faults — not systematic ones.

LESSON · VERIFICATION ≠ VALIDATION

Pentium FDIV, 1994



Hardware ships once.

$$4,195,835 \div 3,145,727$$

CORRECT

1.333820449...

PENTIUM

1.333739068...

FOUND BY

A mathematics professor computing prime numbers
— Thomas Nicely, Lynchburg College, 1994.

WHAT CHANGED

Formal verification of arithmetic units became
standard practice.

THE FLAW

5 of 1,066 cells in the divider's
lookup table were empty

THE ODDS

1 in 9 billion random divisions
affected — Intel's white paper

THE TURN

20 Dec 1994: no-questions-asked
replacement policy

THE BILL

\$475 M pretax charge, January
1995

LESSON · SILICON VERIFICATION

Three failures, three lessons



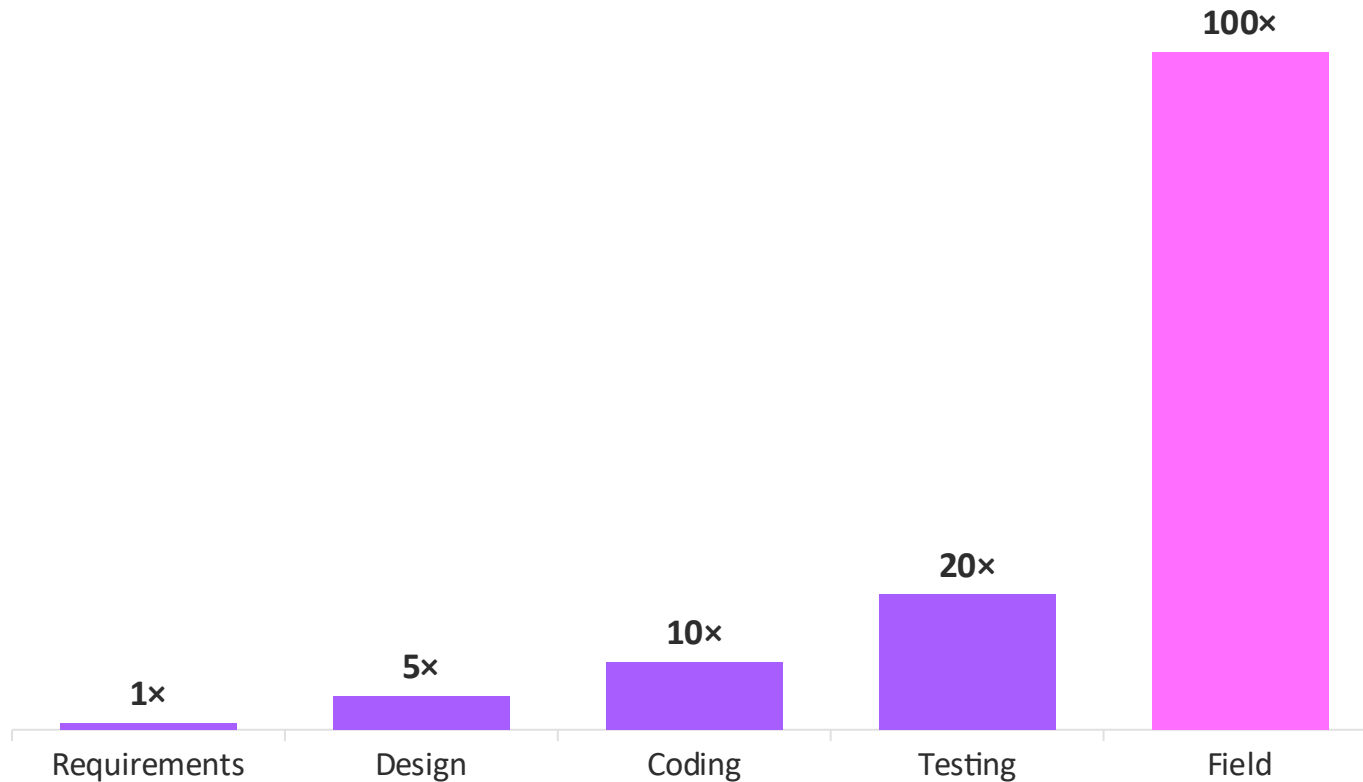
Case	What broke	The test that would have caught it	Cost
Mars Climate Orbiter	Unit mismatch at a software interface — each side correct alone	End-to-end integration test with real trajectory data	\$327.6 M
Ariane 5 flight 501	Overflow in code inherited from another vehicle	Validation against Ariane 5 trajectory data	>\$370 M
Pentium FDIV	Five empty cells in a hardware lookup table	Exhaustive (formal) verification of the divider	\$475 M

All three escaped into the field. None was caught where it would have been cheap.

The later you find it, the more it costs



Relative cost of fixing a defect, by the phase in which it is found



RULE OF THUMB

Each phase a defect survives multiplies the cost of fixing it several-fold — about two orders of magnitude between requirements and the field.

Exact multipliers vary by study; the shape of the curve does not.

SO

“Shift left”: move detection toward the start of the cycle — where fixing is a sentence, not a recall.



“Program testing can be used to show the presence of bugs, but never to show their absence!”

— Edsger W. Dijkstra, 1970

Testing measures risk. It does not remove it.

A green test suite is not a bug-free product — it is a product whose known risks are under control.

What a good test buys you



FOR THE ENGINEER

Speed and courage

- › Feedback in minutes, not weeks
- › Freedom to change code — the safety net will tell you
- › Documentation that cannot lie: a test describes what the system really does

FOR THE PRODUCT

Stability

- › Regression safety: what worked keeps working
- › Known risk instead of unknown risk
- › Faster releases with fewer surprises

FOR THE BUSINESS

Evidence

- › Proof for certification — and for customers
- › Cheaper failures, found on the left of the curve
- › Predictable delivery

Next: how we do it — software testing.

Agenda · 02



- 01 Why testing matters
- 02 Software testing: the essentials**
- 03 How we do it: the QNTAF strategy
- 04 Hardware testing: the silicon lifecycle
- 05 Wrap-up & discussion



Verification vs validation



Two different questions — and Ariane 5 answered only one of them

VERIFICATION

Are we building it right?

- › Does the product meet its specification?
- › Reviews, analysis and tests against the requirements
- › Ariane 5's inertial unit: verified — against Ariane 4's specification

VALIDATION

Are we building the right thing?

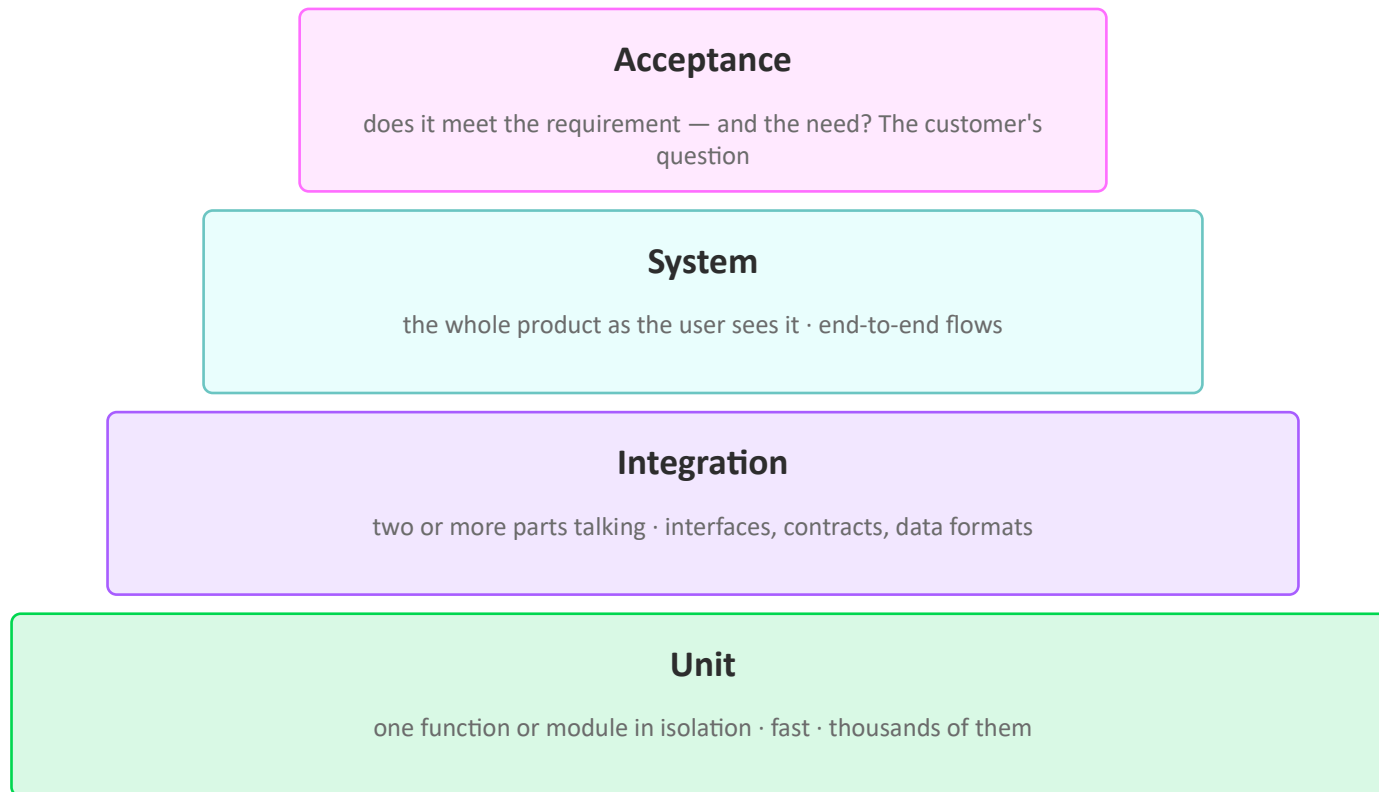
- › Does the specification meet the real need, in the real environment?
- › Trials with real data, real users, real trajectories
- › Ariane 5: never validated with Ariane 5 trajectory data

Both are needed — and passing one says nothing about the other.

Test levels



Each level sees something the others cannot



THE PYRAMID RULE

Many fast tests below, few slow tests above. Invert it and feedback takes hours — and nobody runs the suite.

OUR THREE STORIES

MCO died at the integration seam — every unit passed.

Ariane died at acceptance — the system met the wrong specification.

FDIV: the unit itself was wrong — five cells.

Anatomy of a test — and five ways a test lies



A test must be able to fail. If it cannot, it is not a test.

ANATOMY

Arrange · Act · Assert

- › Arrange — put the system in a known state
- › Act — do the one thing under test
- › Assert — compare the outcome with an expected value
- › The baseline rule: if the code returned nothing, an empty result or the wrong value, the test must go red.

FIVE WAYS A TEST LIES — ALL FOUND IN REAL CODE

Asserts presence, not value

```
check "status" exists in result      -- passes for any status
```

Checks the type, not the content

```
check result is a list              -- passes for an empty list
```

A vacuous OR

```
check x == "ok" OR length(x) <= 6   -- true for every short status
```

A no-op update

```
set user.status = "active"           -- it was already active
```

Observes “loading” only after it is done

```
check page is not loading            -- the state you wanted is gone
```

Spot the bug



An access gate, in pseudocode. One test would have caught it — which?

PSEUDOCODE — NO PARTICULAR LANGUAGE

```
LEVELS = { "evaluation": 1, "validation": 2, "certification": 3 }

function allowed(user_level, min_level):
    have = lookup(LEVELS, user_level, default = 1)
    need = lookup(LEVELS, min_level, default = 1)
    return have >= need

-- in a request handler
if not allowed(current_user.level, "certification"):
    reject with 403
```

Read it. Say nothing for a minute. Then tell me what you would test.

THE QUESTION

Something in these lines lets a user into a place they should not be. Where — and what test would have found it before a user did?

HINT

Think about a value that is not in the dictionary.

INTERACTIVE · 4 MIN

Reveal: gates fail closed



An unknown key must produce the most restrictive answer — never the most permissive

THE BUG

```
need = lookup(LEVELS, min_level, default = 1)
-- unknown level -> 1 -> everyone passes
```

THE FIX

```
need = lookup(LEVELS, min_level, default = highest_value(LEVELS))
-- unknown level -> the most restrictive one
```

THE TEST THAT CATCHES IT

```
test "an unknown gate level denies lower levels":
  assert allowed("evaluation", "certifcation") == false  -- note the typo
```

IT WAS REAL

In our frontend's access gate: the default for a missing key was the lowest level.

Fixed with the most restrictive default — plus a test with a realistic typo.

THE RULE

Fail closed. When in doubt, deny.

**The most valuable tests are for the cases that “cannot happen”.
They happen.**

The vocabulary you'll meet (1/2)



Four distinctions — each gets its own slide next

BLACK BOX / WHITE BOX

Testing from the outside — inputs and outputs — versus with knowledge of the internals.

Login: black box sends a wrong password and expects 401; white box tests the branch that compares the bcrypt hash.

STATIC / DYNAMIC

Analysing code without running it versus executing it.

The linter flags an unused variable (static); the Robot test logs in and reads the response (dynamic).

FUNCTIONAL / NON-FUNCTIONAL

What it does versus how well it does it.

Functional: wrong password → 401. Non-functional: the answer arrives in ≤ 300 ms.

COVERAGE

How much of the code — or of the requirements — the tests exercise.

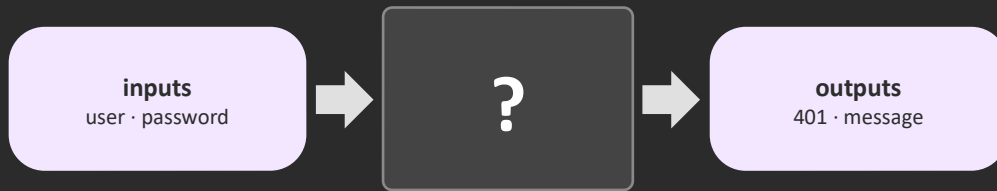
Every line of the login handler ran, yet nobody tried a 300-character username.

Black box vs white box



Two views of the same login handler — and why you need both

BLACK BOX · TESTS FROM THE SPECIFICATION

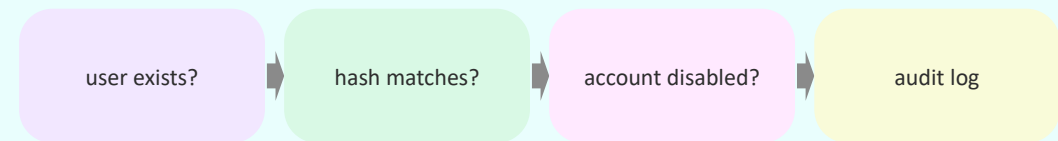


You see only what goes in and what comes out. Tests are written from the requirement — the tester need not read the code.

Finds: misunderstandings of the spec, missing behaviour, wrong messages, the case the developer never imagined.

Misses: the branch nobody reaches from the outside — the error path that only fires when the database is down.

WHITE BOX · TESTS FROM THE STRUCTURE



You see the code. Tests are written from its structure: every branch, every error path, every constant.

Finds: untested branches, dead code, the wrong constant, the missing else.

Misses: what the code should have done and does not — the requirement that was never implemented has no branch to cover.

Complementary, not rivals. Most real suites are grey box: black-box tests written by someone who has read the code.

Static vs dynamic



Without running the code — and by running it

	Static — without running	Dynamic — running it
What it is	reviews, linters, type checkers, light formal analysis	unit, integration and system tests; fuzzing
What it catches	undefined names, type errors, dead code, insecure patterns, complexity	wrong behaviour, integration failures, timing, resource use
What it needs	just the source code	inputs, a running environment, and an oracle for the expected result
Cost and speed	seconds — on every keystroke or every push	minutes to hours — needs a built, running system
Blind spot	logic that is wrong but well-formed	the input nobody wrote

Login: the linter flags the unused `retries` variable in seconds; only the Robot test can find that a wrong password still sets a cookie.

Order: static first — cheapest, whole classes of bugs. Dynamic second — the specific behaviour the requirement asks for.

Functional vs non-functional — the login, worked



The functional ones prove the feature exists; the non-functional ones decide whether anyone can live with it

Kind	Requirement	How you test it	Where it lives in our work
Functional	wrong password → 401 and “Invalid username or password.”	API and UI Robot tests expect the code and the message	AC-008 of QNTAF-121 · VVC-144, VVC-31
Functional	an engineer cannot reach an admin-only endpoint	Robot test expects 403	AC-005 · VVC-73
Non-functional · performance	the login answers in ≤ 300 ms, token included	load test: 100 concurrent logins, check the 95th percentile	QNTAF-121’s own performance target
Non-functional · security	the same error for a wrong user and a wrong password — no user enumeration	compare the two responses byte for byte	the wording of AC-008 (“username or password”)
Non-functional · security	passwords stored only as bcrypt hashes	integration test reads the DB row: no plaintext, hash prefix present	QNTAF-121’s tooling note: bcrypt
Non-functional · usability	the form works on a phone screen	UI test at 375 px width	not in this Design Package — yet

Most requirement documents write the first kind and forget the second. Always ask: how fast, how secure, how usable — and how would we test that?

Coverage: what 100 % does — and does not — mean



Four tests, every branch covered, one bug still there

PSEUDOCODE — THE LOGIN HANDLER

```
function login(user, password):  
    if user not in store:           return 401, "Invalid username or password."  
    if not hash_matches(password): return 401, "Invalid username or password."  
    if store[user].disabled:       return 403, "Account disabled."  
    return 200, new_session(user)
```

THE BUG COVERAGE NEVER ASKED ABOUT

A 300-character username is never length-checked in any of these four lines. In the real handler it reaches a layer that cannot take it and the request ends in a 500, not a 401. Every line ran; that input never existed in the suite.

Test	Exit taken
unknown user	1st check → 401
wrong password	2nd check → 401
disabled account	3rd check → 403
happy path	falls through → 200

LINE AND BRANCH COVERAGE: 100 %

THE OTHER COVERAGE

Requirements coverage: does every acceptance criterion have a test? That is what the traceability loop measures — a different animal.

Coverage tells you where you have not looked. It never tells you what you have not thought of.

The vocabulary you'll meet (2/2)



Four words you will hear in every test conversation

REGRESSION

A test that guards something that already worked. Most of a mature suite is regression.

Example: the login test has passed 400 times and still runs on every pull request — the day it fails, something you just changed broke it.

FLAKY

Passes and fails with no code change. Worse than no test: it trains people to ignore red.

Example: a UI test that fails one run in twenty because a page loads slowly. Fix the wait — or delete the test.

TEST ORACLE

How you know the expected answer. The hardest problem in testing when there is no reference model.

Example: for the Pentium divider the oracle was exact arithmetic; for a RISC-V core it is Spike, the reference simulator.

MOCK / STUB

A stand-in for a dependency, so one part can be tested alone with the other's behaviour under control.

Example: test the login handler against a fake user store that returns exactly the user you want — no database needed.

Exercise 1 · Which test catches it?



In pairs · 5 minutes · six defects in the login you already know — name the cheapest test that would have caught each

#	The defect	Your answer
1	An unknown usage level is treated as level 1 — the door opens	
2	A wrong password is rejected — but the session cookie is set anyway	
3	Unknown user and wrong password return different messages	
4	A 300-character username returns 500 instead of 401	
5	Passwords land in the database in plain text — bcrypt never wired in	
6	Login takes 900 ms with 50 users at once; the target is ≤ 300 ms	

Every one of these is real or one step from real. Which one would you have written a test for?

EXERCISE · 10 MIN

YOUR TOOLBOX

review · static analysis · unit test · integration test · API / system test · load test

RULES

One answer per defect: the cheapest place it could have been caught.

Then tag it with today's words: black or white box? functional or non-functional?

LISTEN FOR

“A system test” for everything — cheaper nets exist.

Defects 3 and 4: no test in the suite fails today. How do you even find them?

Exercise 1 · The cheapest net for each



All six are cheaper than the field — and two of them need a test nobody writes from coverage alone

The defect	Cheapest test that catches it	In today's words
1 · unknown level → level 1	one unit test with an unknown level, expecting denial — the fix-first test of the reveal	unit · white box · functional
2 · cookie set on a wrong password	API test asserting no session cookie — VVC-144 does exactly this	system (API) · black box · functional
3 · two different error messages	API test sending both cases and comparing the responses byte for byte	system (API) · black box · security
4 · 300-character username → 500	a boundary test at unit level — or a fuzzer, which finds it without you thinking of it	unit · black box · robustness
5 · plain-text passwords	integration test reading the database row: hash prefix present, no plaintext	integration · white box · non-functional
6 · 900 ms under load	load test with concurrent logins against the ≤ 300 ms target of the Design Package	system · black box · non-functional

The level tells you the price. The vocabulary tells you the kind. Defects 3 and 4 tell you that coverage cannot replace thinking.

Test-driven development: write the test first



Red → green → refactor — the test is the specification you write before the code



Not a religion — a habit.

It pays back the first day you refactor and the suite tells you exactly what you broke.

WHY IT CHANGES THE CODE

You design the interface before the implementation — untestable code never gets written.

Every line exists because a test asked for it.

BDD IS TDD ONE LEVEL UP

The Then of an acceptance criterion is written before the feature — the same discipline, at the requirement level.

Static analysis: finding bugs without running anything



Tests find instances of a bug; static analysis finds the whole class

LINTERS & TYPE CHECKERS

As you type

Undefined names, unused variables, impossible types, unreachable branches — flagged in the editor before the code ever runs.

QUALITY GATES

On every pull request

SonarQube-style analysis: duplication, cognitive complexity, hard-coded secrets, security hotspots. Ours runs on every pull request and turns the check red when new code fails the bar.

FORMAL METHODS, LIGHT

Proven, not sampled

Null dereferences, overflows and unreachable code proven on every path — the idea the hardware block will call formal verification.

Static analysis finds classes of bugs; tests find instances. You want both — and static is the cheaper one to run first.

Beyond the tests you wrote: mutation testing and fuzzing



Both attack the same weakness — tests only check what a human imagined

MUTATION TESTING — TEST THE TESTS

Break the code on purpose

A tool flips `>=` to `>`, deletes a line, negates a condition — hundreds of mutants. Rerun the suite: a mutant that survives is a test that cannot fail. The mutation score measures your tests, not your code.

FUZZING — INPUTS YOU DID NOT THINK OF

Millions of inputs, no opinions

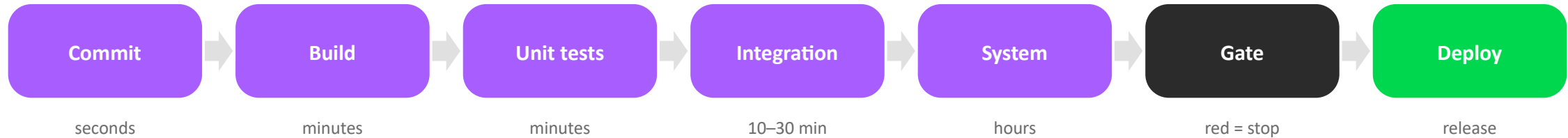
Generate random or grammar-driven inputs, run, and watch for crashes, hangs and memory errors. Heartbleed was one of the bugs a fuzzer found. In hardware the same idea is called constrained-random.

The rule again: a test must be able to fail. Mutation testing is how you check that it can.

Where tests run: the CI/CD pipeline



Every change walks the same path. Gates decide whether it continues.



GATES

A gate is an automatic decision: red means the change does not move.

Nobody argues with a gate at 3 a.m.

SHIFT LEFT

The cheaper the phase, the more tests it should carry — which is why unit tests run first and most often.

Block 1's cost curve, turned into architecture.

From requirement to test: BDD



A requirement written so that it can be executed

GIVEN

the login API is called

WHEN

a user provides an incorrect username or password

THEN

login is rejected with 401 Unauthorized, an error message is returned, and no JWT cookies are set

Acceptance criterion AC-008 · Design Package QNTAF-121 — real, from our Jira.

WHY THIS SHAPE

Given = the state (Arrange). When = the action (Act). Then = the expected outcome (Assert).

A BDD criterion is a test written in the customer's language.

THE RULE

If you cannot write the Then, you do not yet have a requirement.

Exercise 2 · Write an acceptance criterion



In pairs · 5 minutes to write · three pairs read out

THE REQUIREMENT

REQ · A logged-in user can change their own password.

GIVEN

WHEN

THEN

RULES

One criterion, three lines: Given / When / Then.

The Then must be something a machine could check — a state, a message, a code.

Bonus: the negative case (wrong current password).

LISTEN FOR

A vague Then (“it works”).

Two actions in one When.

A Then that forgets what must NOT happen.

EXERCISE · 12 MIN

Exercise 2 · Debrief



One that would survive a review — and the three traps

GIVEN

a logged-in user whose current password is P1

WHEN

they submit P1 and a new password P2 that meets the policy

THEN

the request succeeds (200), a confirmation is shown, the session stays valid — and P2 logs in while P1 is rejected with 401

Negative case · When the current password is wrong → Then 401, the password is unchanged (P1 still logs in), and the attempt is logged.

THREE TRAPS

“Then it works” — not checkable. Name the state, the message, the code.

Two actions in one When — that is two criteria.

Forgetting the absence: the old password must stop working. The best Thens say what must NOT happen.

WHY WE INSIST

Every criterion you just wrote is a test somebody can automate tomorrow. That is the whole point of the format.

Agenda · 03



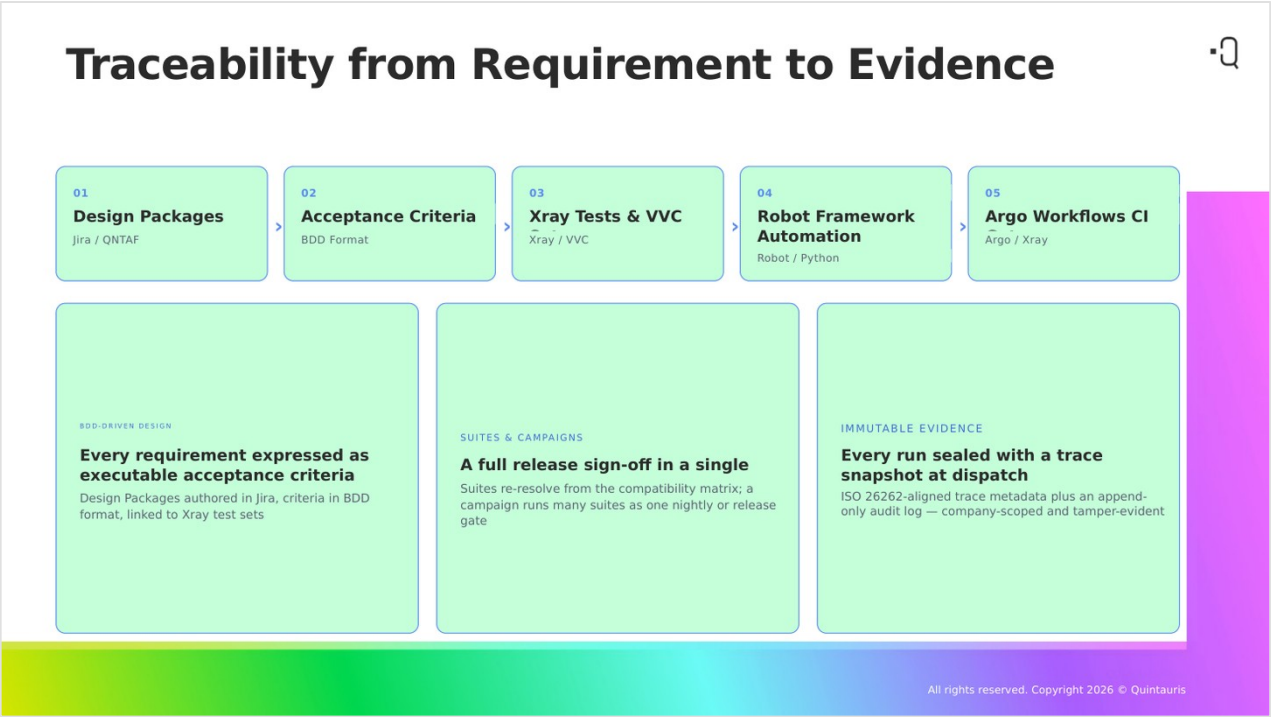
- 01 Why testing matters
- 02 Software testing: the essentials
- 03 How we do it: the QNTAF strategy**
- 04 Hardware testing: the silicon lifecycle
- 05 Wrap-up & discussion



Wednesday you saw what. Today: why.



Enrique showed the chain. This block is about why it is shaped like that.



Session 3, Wednesday — “Traceability from Requirement to Evidence”

THREE WHYS

Why every requirement is traced to a test — and back.
Why tests live in layers with different owners.
Why the release is decided by gates, not by meetings.

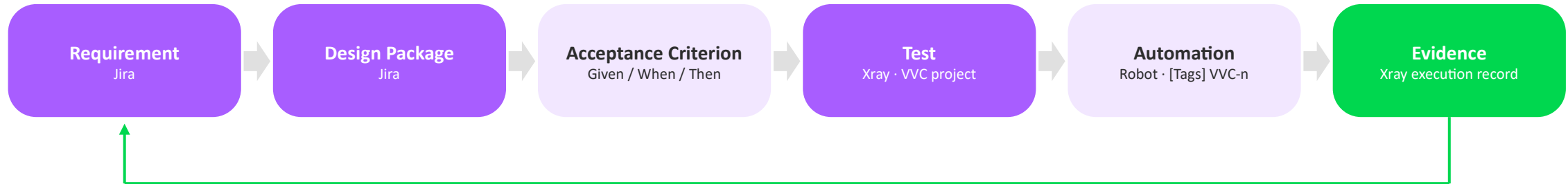
AND ONE EXAMPLE

One real requirement — the login criterion you just saw — followed from Jira to the recorded evidence.

The traceability loop



Enrique's chain, plus the arrow he did not draw: the last link points back to the requirement



THE TWO QUESTIONS CERTIFICATION ASKS

“Show me the test for this requirement.” · “Show me the requirement for this test.”

Both answered in one click — because the link is data, not memory.

THE LINK IS A TAG

[Tags] VVC-144 in the Robot test is the Xray key.
Same words in three places: criterion, test, code.

Test layers — and who owns them



The people who write the code do not sign off the system. That independence is the point.

Layer	Owner	Tools	When it runs
Unit	Developers	pytest · vitest	every commit · seconds
Integration	Developers	pytest + a real PostgreSQL in a container	every commit · minutes
System	V&V (design) + automation engineer	Robot Framework · Selenium · RequestsLibrary	pre-merge subset on every pull request · full suite before release
Acceptance · certification	Customer · certification body	Reports generated from the same evidence	at milestones

Tests are part of “done”: a feature without its unit and integration tests is not finished.

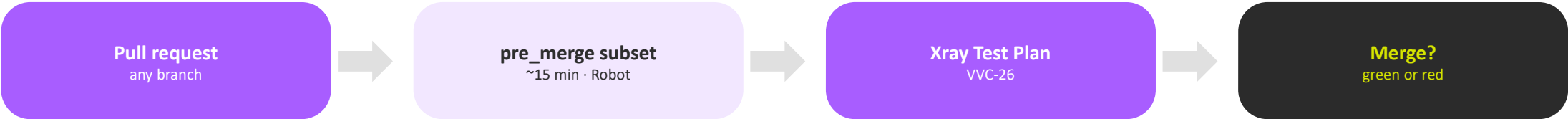
The system layer is written from the acceptance criteria, not from the code — it checks what was asked, not what was built.

Two gates, two test plans

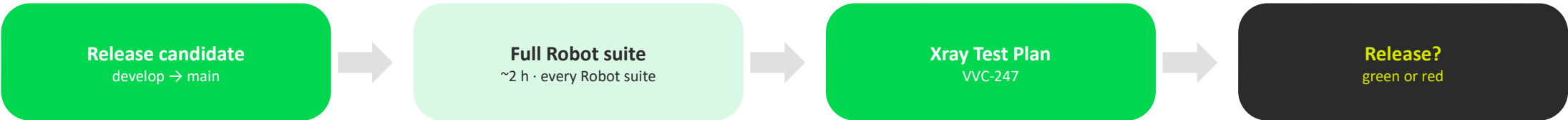


Fast on every pull request; complete before every release

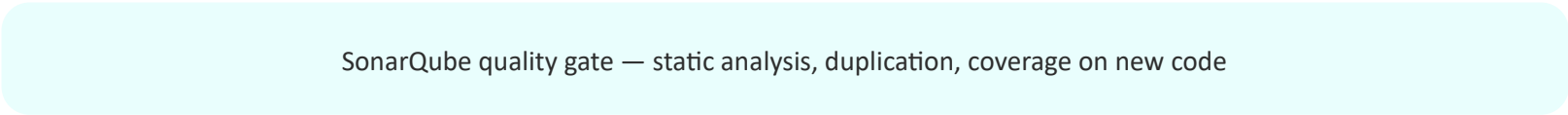
ON EVERY PULL REQUEST



BEFORE EVERY RELEASE



ON EVERY PULL REQUEST, TOO



THE LOOP CLOSES

Results flow back into Xray; the gate reads the plan, humans read the gate.

VVC: one verification project for every product



Tests are organised around the product under test — not around the team that wrote the code

SYSTEM UNDER TEST

One project, many products

Every test names the product it verifies. Today QNTAF; tomorrow the next product — same project, same rules.

TEST TYPE = AUTOMATION SIGNAL

Manual or Automated

Manual: specified, not yet automated. Automated: a Robot test with this key exists. No labels, no spreadsheets — the type is the truth.

ONE TEST SET PER DESIGN PACKAGE

Coverage is a query

The set covers the requirement; the tests cover its criteria. “Is this covered?” is answered by Jira, not by memory.

PRECONDITIONS

Shared setup, written once

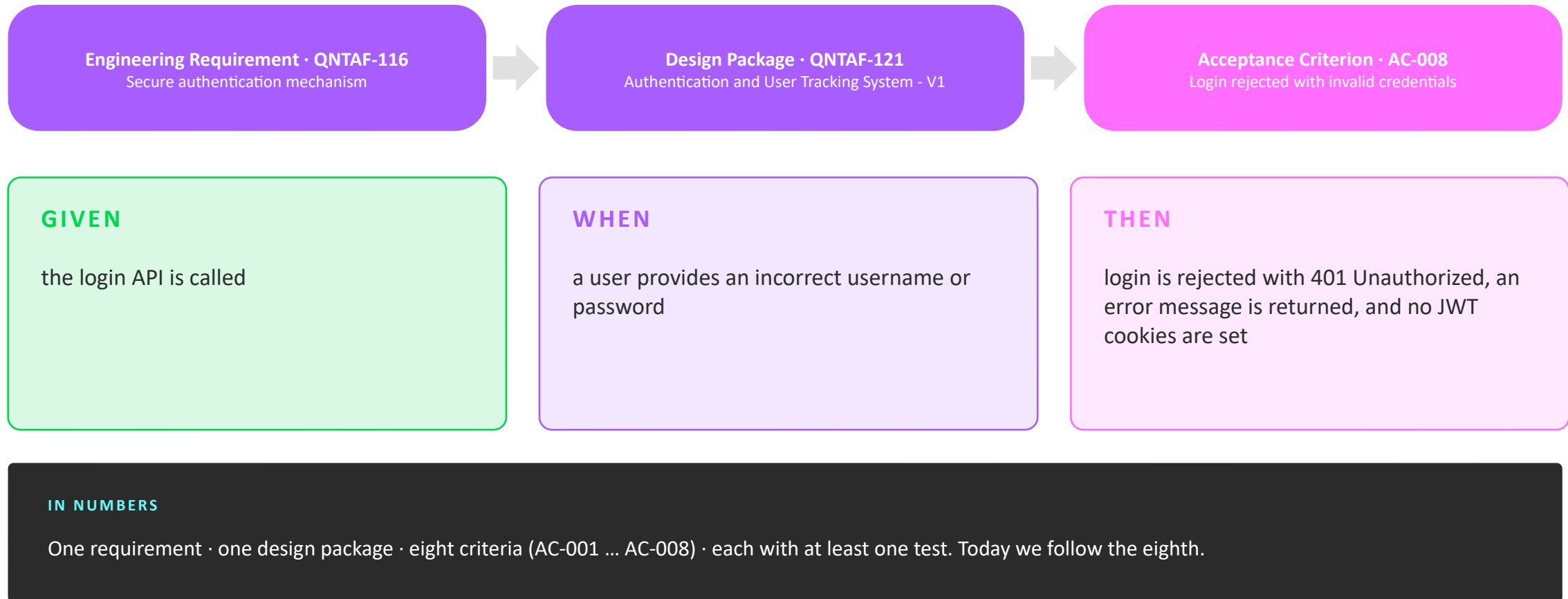
Users, data, environment — linked to every test that needs them, so tests stay short and honest.

One place to ask: what is verified, how, and with what result?

Worked example 1/4 — the requirement



From an engineering requirement to an executable criterion. All real, all in Jira.



Worked example 2/4 — the API test



VVC-144 · runs on every pull request, inside the pre-merge gate

XRAY TEST · VVC-144

Login Rejected With Invalid Credentials

- › Step 1 · POST /login with a wrong username → 401, an error message, no JWT cookies
- › Step 2 · POST /login with a wrong password → 401, an error message, no JWT cookies
- › Type: Automated · Test Set VVC-41 · traces to AC-008

ROBOT FRAMEWORK · API.ROBOT · EXCERPT (STEP 1; STEP 2 IS SYMMETRIC)

```
Login Rejected With Invalid Credentials
[Tags]    authentication    VVC-144    pre_merge
VAR    &{headers}=    Content-Type=application/json
VAR    &{wrong_user_body}=    username=wrong_user    password=user
${wrong_user_response}=    POST On Session    ${API_SESSION}    /api/login
...    json=${wrong_user_body}    headers=${headers}    expected_status=any
Should Be Equal As Numbers    ${wrong_user_response.status_code}    401
...    Wrong username should return 401 Unauthorized
${access_cookie}=    Evaluate
...    ${wrong_user_response.cookies.get('access_token_cookie')}
Should Be Equal    ${access_cookie}    ${None}
...    No JWT cookies should be set for failed login
```

Same words in the criterion, in Xray and in the code: “401”, “no JWT cookies”.

The tag VVC-144 is the only link needed — the rest is a query.

Worked example 3/4 — the UI test



VVC-31 · runs in the full suite before every release

XRAY TEST · VVC-31

Incorrect Login User

- › Step 1 · Navigate to the Login page → page displayed
- › Step 2 · Enter invalid credentials, click Login → inline error “Invalid username or password.”
- › Step 3 · User remains on the login page → no session created
- › Type: Automated · Test Set VVC-41 · traces to AC-008

ROBOT FRAMEWORK · LOGIN.ROBOT · AS IN THE REPOSITORY (ONE LONG LINE WRAPPED)

```
Incorrect Login User
[Documentation]    Test the login functionality navigating through the frontend
...    with incorrect user credentials.
[Tags]    VVC-31
Login User    wrong_user    wrong_password
Wait Until Element Is Visible    xpath=//p[@role='alert']    timeout=5s
${error_text}=    Get Text    xpath=//p[@role='alert']
Should Be Equal As Strings    ${error_text}    Invalid username or password.
Element Should Be Visible    css=#LoginPage .login-card
```

One criterion, two layers.

The API test proves the contract; the UI test proves what the user actually sees. Both must be green for AC-008 to count as verified.

Worked example 4/4 — the evidence



The requirement is not “done”. It is proven — with a timestamp.



WHAT AN AUDITOR SEES

Requirement → criterion → test → execution → result — each one a link.

No slides, no promises: records.

Two evidence chains, one method: Xray records what the tests proved; QNTAF's trace snapshot records what each benchmark job ran.

AND WHEN IT FAILS

A red test is either a product bug or a stale specification.

Deciding which — “spec wrong or product wrong?” — is the daily work of verification. The test is never edited just to pass.

Exercise 3 · Your test plan for a login page



In pairs · 5 minutes · list every test you would run, numbered

Sign in

Sign in

[Forgot password?](#)

— or — Sign in with your company account

WHAT TO COVER

Functional: what must work, what must be refused.

Security: what must never be possible.

Robustness: what happens at the edges — empty, huge, twice, expired.

After login: what changes elsewhere — sessions, logs, permissions.

HOW

Number them. Aim for ten. You will compare with our real list in five minutes.

EXERCISE · 15 MIN

Exercise 3 · What we actually test



Test Set VVC-41 — the twelve automated tests behind Design Package QNTAF-121, one or more per criterion

AC	Test	Name	What it checks
AC-001	VVC-69	Login And Protected Route Access	a valid JWT is issued for correct credentials (API)
AC-001	VVC-29	Correct Login And Logout Admin User	login and logout through the UI — session established, then terminated
AC-002	VVC-70	Access protected route with valid token	protected endpoints: 200 with a token, 401 without
AC-003	VVC-71	Access Denial Scenarios	a malformed refresh token is refused with 401
AC-004	VVC-72	Audit Log Records User Actions	running a test is written to the audit log with user and timestamp
AC-005	VVC-73	Role-based access control	an engineer gets 403 on /admin endpoints
AC-005	VVC-35	Access Admin Page Without Admin Rights	the Admin page is not rendered for an engineer — even by direct URL
AC-006	VVC-74	Expired access token after timeout	the access token dies after 15 min; a valid refresh token renews it
AC-007	VVC-82	Refresh token is expired	expired refresh token → logged out, must sign in again
AC-008	VVC-144	Login Rejected With Invalid Credentials	401 and no cookies for a wrong user or a wrong password (API)
AC-008	VVC-31	Incorrect Login User	the inline error message; still on the login page (UI)
AC-008	VVC-32	Empty Fields Login User	empty fields are refused before anything is sent

...and more login-related tests live in other Design Packages: expired-session logout, password policy, disabled accounts, refresh-token whitelist, the redesigned login page.

What it gives us — and what it costs



Honest accounting

IT GIVES US

- › Drift detection: when the product and its specification disagree, a test says so first
- › Release confidence: the gate is green or it is not
- › Certification evidence, generated as a by-product of daily work
- › Onboarding: the tests are the most honest documentation we have
- › AI-assisted audits of requirements and tests — the structure is machine-readable

IT COSTS US

- › Maintenance: every UI change touches a test
- › Flaky UI tests — the constant enemy; they get fixed or deleted, never ignored
- › Discipline: a criterion you cannot execute is sent back
- › Time up front: writing the Then before the code feels slow — until the first regression it catches

Five principles



If you remember nothing else from this block

1

Test at every level

each one sees what the others miss

2

A test must be able to fail

or it is decoration

3

Write requirements you can execute

Given, When, Then

4

Trace both ways

requirement → test → requirement, as data

5

Gate on evidence

the pipeline decides; people read the decision

Agenda · 04



- 01 Why testing matters
- 02 Software testing: the essentials
- 03 How we do it: the QNTAF strategy
- 04 Hardware testing: the silicon lifecycle**
- 05 Wrap-up & discussion



Why silicon is different



You cannot patch a chip. Everything about hardware verification follows from that.

NO PATCHES

Fixed by a respin

A bug in silicon is fixed by a new silicon revision. Months, not minutes — and the old parts are already in the field.

MASKS COST MILLIONS

Before the first wafer

A mask set at an advanced node costs millions of dollars — paid before a single die exists.

VERIFICATION ≈ DESIGN

About half the effort — often more

Industry surveys (Wilson Research Group, 2018–2024) put verification at roughly half of total project effort, with verification headcount now matching design headcount — and up to 5:1 on processor projects.

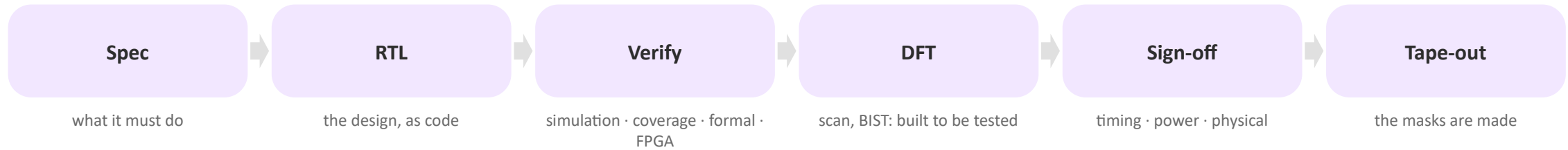
Which is why FDIV cost \$475 M — and why the silicon industry took formal methods seriously first.

The silicon lifecycle

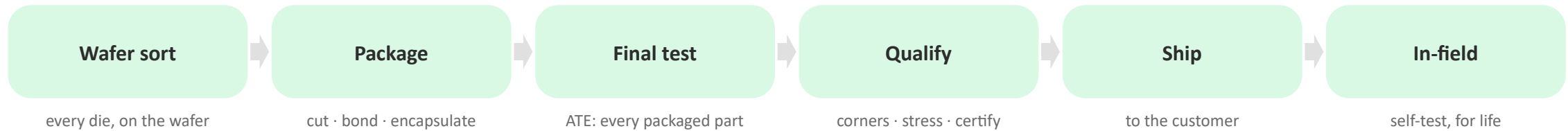


Twelve stages. Testing happens in all of them — differently.

PRE-SILICON · FIND THE DESIGN BUG BEFORE THE MASKS



POST-SILICON · FIND THE MANUFACTURING DEFECT BEFORE THE CUSTOMER



Two different problems, two different toolboxes — and testing in every stage.

Pre-silicon verification in one slide



You will do all of this hands-on in sessions 6 and 7 — today, the map



SIMULATION

Software that tests hardware

Stimulus → the RTL under test → a checker against a reference model.
Directed cases you write; constrained-random cases the tool invents.
SystemVerilog + UVM, or Python with cocotb.

COVERAGE

When are we done?

Code coverage: which lines and branches ran. Functional coverage: which scenarios were observed. The verification plan maps features → coverpoints → tests; closure is a number, not a feeling.

FORMAL

Proving, not sampling

Assertions explored over every reachable state: the bug cannot exist, or here is the exact trace. Used on critical blocks — arithmetic units after FDIV, protocols, safety logic.

FPGA PROTOTYPE · EMULATION

Real software before real silicon

The RTL on an FPGA runs at MHz: boot an OS, run the benchmark, find the bug that appears after a billion cycles. What it cannot see: timing at speed, power, analogue.

Tomorrow and Wednesday: NEORV32 and CVA6 on the Genesys 2 — this slide, with your hands.

Design for Test: building the chip so it can be tested



The bridge from finding design bugs to finding manufacturing defects



SCAN CHAINS

Every flip-flop, observable

In test mode the flip-flops link into one long shift register. Shift a state in, clock once, shift the result out.

ATPG

Patterns, generated

Automatic Test Pattern Generation computes the vectors that detect modelled faults — stuck-at, transition — and reports fault coverage.

BIST

The chip tests itself

Memory BIST and logic BIST use on-chip generators and checkers — at the factory and, later, in the field.

JTAG · IEEE 1149.1

The four-wire door

The test access port: boundary scan for boards — and the debug interface you will use with the FPGA.

DFT costs area and a little speed. Nobody skips it: an untestable chip cannot be sold.

Post-silicon: wafer sort and final test



Every single chip is tested. Twice.



WAFAER SORT

Every die, still on the wafer

A probe card touches each die; the tester runs scan patterns and parametric checks. Bad dice are inked out and never packaged.

THE ATE

Automatic Test Equipment — Advantest, Teradyne: multi-million-dollar machines whose time is billed by the second. Test programs are engineered for coverage per millisecond.

FINAL TEST

Every packaged part

A handler feeds parts to the tester on a load board. Structural, functional and parametric tests — and speed binning.

TWO DIFFERENT QUESTIONS

Validation vs production test

Validation asks “does the design work?” — a lab, a few parts, weeks. Production test asks “is this part defective?” — the ATE, every part, seconds.

What it looks like



A wafer, a packaged part, and the machine that tests them

Spec

RTL

Verify

DFT

Sign-off

Tape-out

Wafer sort

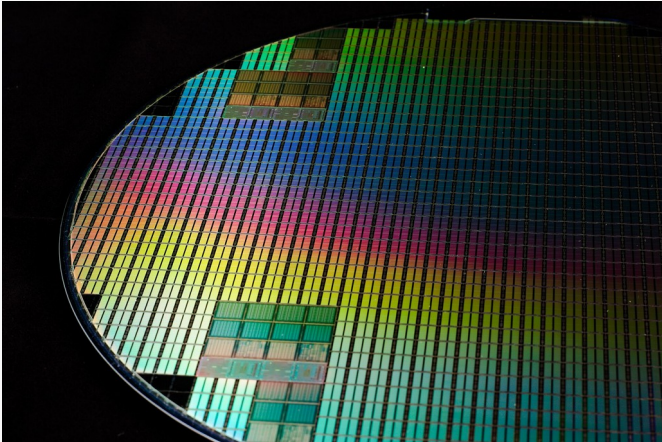
Package

Final test

Qualify

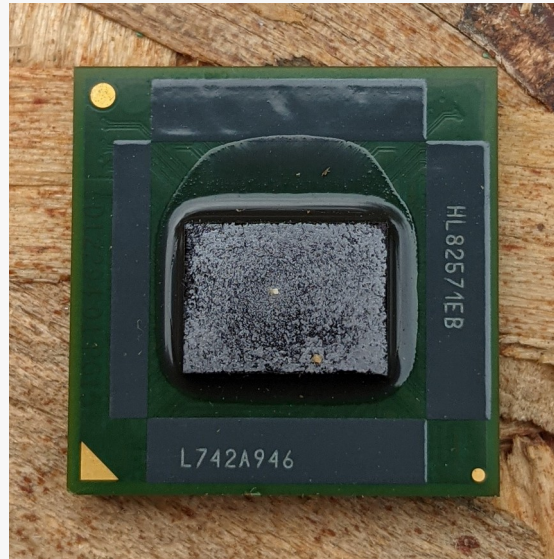
Ship

In-field



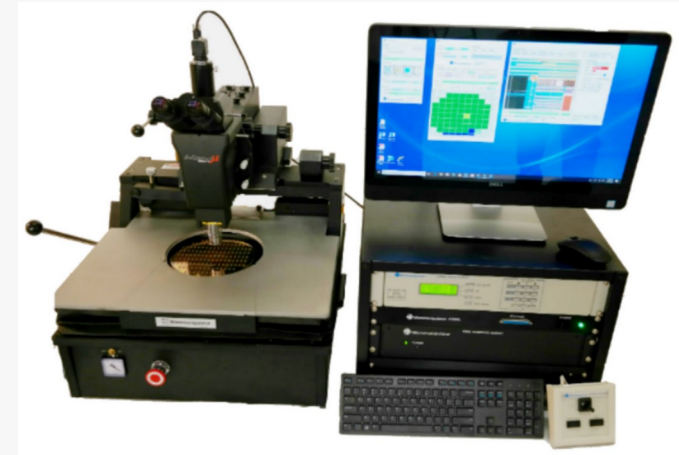
A silicon wafer — hundreds of identical dies, still uncut

Photo: Enrique Jiménez · Wikimedia Commons · CC BY-SA 2.0



A packaged part — the die cut, bonded and encapsulated (BGA)

Photo: Cole L · Wikimedia Commons · CC BY-SA 2.0



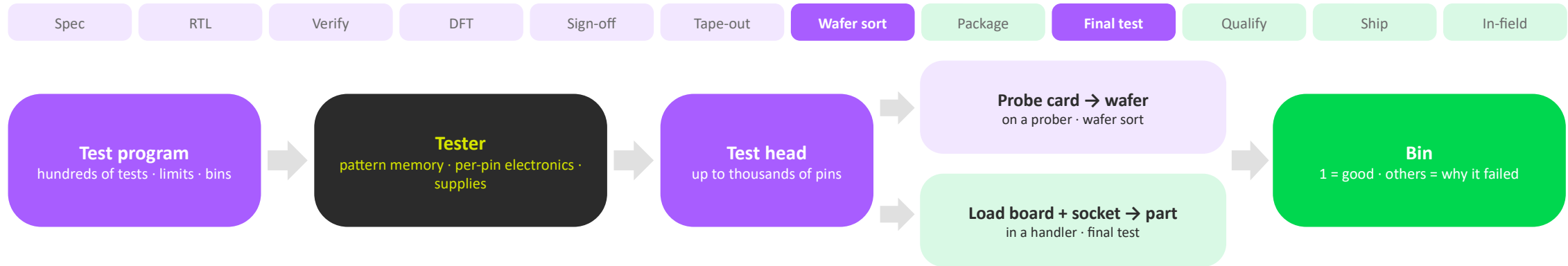
A probe station: the wafer under the needles. A production ATE cell is this, plus a tester with thousands of pins and a handler

Photo: Rgarrihan · Wikimedia Commons · CC BY-SA 4.0

Inside the test cell



How a chip is tested in a few seconds — the hardware and the program



THE TEST PROGRAM

Hundreds of tests in a fixed order — cheapest and most-failing first, so a bad part is rejected in milliseconds.

Every measurement gets limits; every part gets a bin.

TEST TIME IS MONEY

A few seconds per part × millions of parts: programs are engineered for coverage per millisecond.

Multi-site: 4, 8, 16 ... parts tested in parallel on one tester to cut the cost per part.

What the test program checks



Six kinds of test, from milliseconds to seconds



Test class	What it catches	Notes
Continuity · opens / shorts	broken bonds, bent or shorted pins	first and fastest — milliseconds
DC parametrics · leakage, IDD, levels	process defects, weak or leaky transistors	measured per pin and per supply
Scan / ATPG · stuck-at, at-speed	structural manufacturing defects in the logic	the workhorse — most coverage per second
Memory BIST	defective SRAM cells — repaired with spare rows and columns	on-chip engines, all memories in parallel
Functional	boot, checksum, interfaces — what structural tests miss	slow and hard to grade; used sparingly
Speed binning	the maximum frequency each part actually reaches	the same die sold at different prices

Structural tests find defects; functional tests find surprises. Production test leans on the first and keeps a little of the second.

Exercise 4 · Bin the parts



Six parts, one test program, five minutes — sort each part into a bin

Spec

RTL

Verify

DFT

Sign-off

Tape-out

Wafer sort

Package

Final test

Qualify

Ship

In-field

Part	Continuity	Standby leakage · limit ≤ 50 µA	Scan patterns	Fmax · bin 1 ≥ 1.0 GHz · bin 2 ≥ 0.8 GHz	Dynamic IDD · limit ≤ 120 mA
A	pass	12 µA	all pass	1.15 GHz	98 mA
B	pass	48 µA	all pass	1.02 GHz	118 mA
C	FAIL — pin 17 open	not run	not run	not run	not run
D	pass	9 µA	all pass	0.86 GHz	90 mA
E	pass	210 µA	all pass	1.10 GHz	101 mA
F	pass	11 µA	3 patterns FAIL	1.10 GHz	99 mA

BIN 1 · GOOD @ 1 GHZ

BIN 2 · GOOD @ 0.8 GHZ

BIN 4 · SCAN FAIL

BIN 5 · PARAMETRIC FAIL

BIN 7 · CONTINUITY FAIL

Which part would you stop testing first, and why? · Which part passes every limit but still worries you?

EXERCISE · 8 MIN

Exercise 4 · The bins — and the part that worries us



The program stopped on C in milliseconds; B passed everything and is still the one to watch



Part	Bin	Why
A	1	fast, healthy, and centred in the population
B	1 — by the limits	leakage 48 of 50, IDD 118 of 120, Fmax barely 1.02: at the edge of every limit at once — an outlier. Outlier screening would reject it.
C	7	continuity failed — a hard stop: with an open pin nothing else can be measured, so the program ended after the first millisecond test
D	2	everything healthy, just slower — the same die at a lower price
E	5	leakage 4× the limit: a process defect. Scan and speed pass; the part still fails
F	4	three scan patterns fail: a structural defect somewhere in the logic

Order matters: C cost milliseconds because continuity runs first and is a hard stop.
B is why automotive adds outlier screening — next slide.

Yield, escapes and the numbers



Why 99 % fault coverage is not enough for a car



YIELD

Good dice ÷ all dice

Starts low on a new product and climbs as process and design are tuned. Every point of yield is money.

ESCAPES → DPPM

Defective parts that pass

Defect level $\approx 1 - Y^{(1 - C)}$. At 90 % yield, 99 % fault coverage leaves ~1,000 DPPM; 99.9 % leaves ~100. Single digits need more than coverage.

OVERKILL

Good parts that fail

Measurement noise and tight guard-bands reject good silicon. Yield lost at the tester costs as much as yield lost in the fab.

OUTLIER SCREENING

Automotive's extra net

Part Average Testing (AEC-Q001): a part that passes every limit but sits far from its population is rejected anyway.

Single-digit DPPM is not one test — it is coverage, guard-bands and outlier screening, applied to every part.

First silicon: bring-up, characterisation, qualification



The loop closes: the software you ran on the FPGA now runs on the chip



BRING-UP

Does it boot?

First samples on a lab board: JTAG in, clock on.
The same firmware and tests from the FPGA stage
— a new target, not a new suite.

CHARACTERISATION

Where does it stop working?

Sweep voltage, temperature and process corners
— PVT. The answer becomes the datasheet.

QUALIFICATION

Will it last?

Stress beyond the specification to prove lifetime:
temperature cycling, humidity, burn-in.
Automotive: AEC-Q100.

QEMU → FPGA → silicon: one test flow, three targets. That is why we build tests to be target-independent.

In the field: testing never stops



A safety-critical chip tests itself for its whole life



LBIST AT BOOT

A warning light, not an accident

Logic self-test runs every time the car starts. A fault found here is a diagnostic, not a crash.

ECC & LOCKSTEP

Detect while running

Memories correct single-bit errors; two cores run the same code and compare every cycle.

ISO 26262

Diagnostic coverage

Functional safety asks: what fraction of possible faults would be detected in time? ASIL D means almost all of them.

Determinism, lockstep, ECC — the platform requirements you saw on Wednesday are in-field verification.

The lifecycle — and what a bug costs at each stage



Block 1's curve, in silicon

Stage	What is tested	How	A bug found here costs
Spec · RTL	the design intent	reviews, lint, simulation	an edit
Verification	behaviour against the spec	simulation, formal, FPGA	a re-run: hours to days
Tape-out	— the masks are made —	—	a respin: months and millions
Wafer sort · final test	each manufactured die	ATE, scan, parametric	a discarded part: cents to dollars
Qualification	lifetime and corners	stress, PVT sweeps	a delayed launch
In-field	the deployed chip	self-test, ECC, lockstep	a recall — or worse

Same curve as software — with a cliff at tape-out.

Agenda · 05



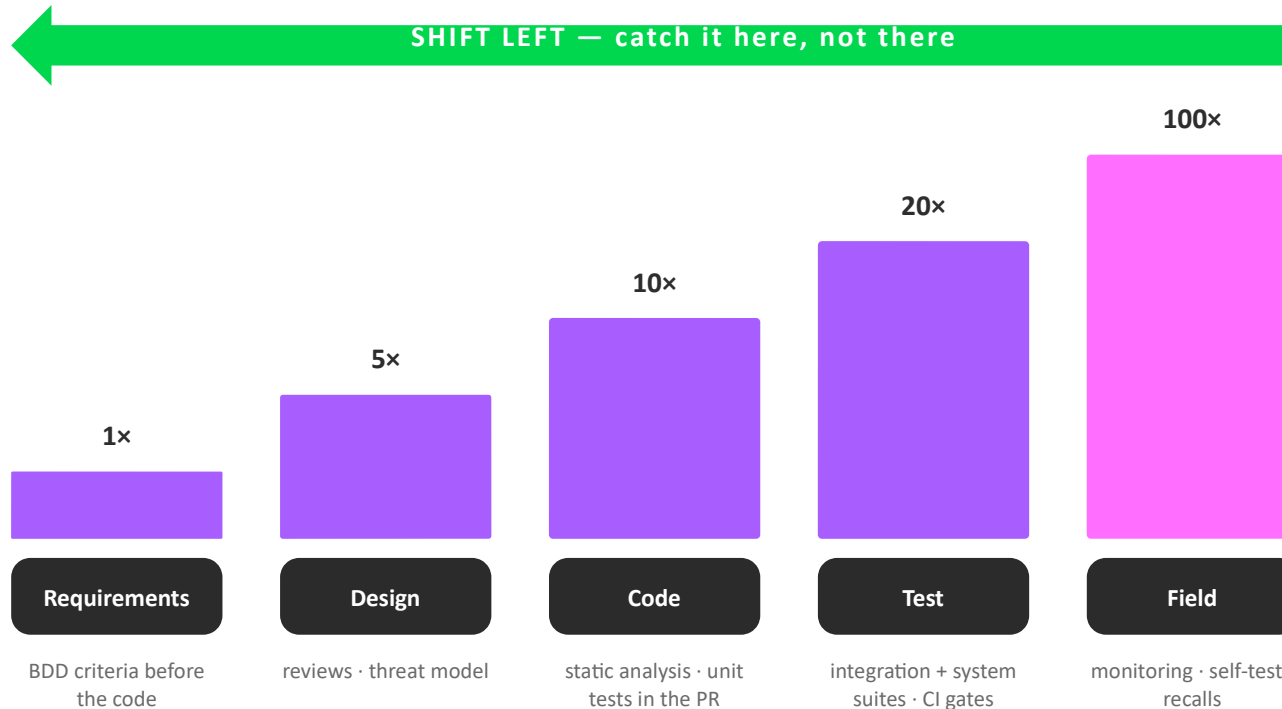
- 01 Why testing matters
- 02 Software testing: the essentials
- 03 How we do it: the QNTAF strategy
- 04 Hardware testing: the silicon lifecycle
- 05** Wrap-up & discussion



Shift left: find it where it is cheap



Every phase a defect survives multiplies the cost of fixing it — so move detection to the left



Relative cost of a fix — indicative multipliers, bars not to scale

Testing late is not testing badly — it is testing expensively.

WHY IT SAVES MONEY

Block 1's curve: two to five times per phase, two orders of magnitude end to end.

MCO, Ariane 501 and FDIV were all found in the field — the far right of this chart.

IN SILICON

The cliff is tape-out: after the masks, a design bug is a respin of months and millions.

THE RULE

Detect at the cheapest phase that can detect it.

One idea to take home



Requirement → test → evidence.
Software or silicon, the loop is the same.

SOFTWARE

Given / When / Then → Robot test → Xray result

QNTAF

Design Package → VVC test → CI gate → Xray execution

SILICON

vPlan → coverpoint → simulation · spec → ATPG pattern → ATE
result

Tomorrow you put a core on an FPGA. At every step, ask: how do I know this works?

That question is the whole craft.

Thank you for
your Attention!



·QUINTAURIS